

From a one-line idea to a published app.

A Claude Code plugin that takes **Power Apps Canvas + Power Automate** apps, on SQL Server or Dataverse, through 10 guided stages. Specialist agents build, validate and prove what they did; you approve, paste into the environment and publish.

10 guided stages	8 specialist agents	25 ready Canvas components	34 Power Automate blocks
----------------------------	-------------------------------	--------------------------------------	------------------------------------

WHY I BUILT IT

I started in Power BI and moved on to Power Apps, Power Automate, RPA and process automation. On every project the same problems came back, and they were almost always solved the same way. I turned that way into rules, and the rules into a kit that Claude Code follows from start to finish.

The goal is simple: less rework, the same standard across every app on the team, and every delivery with proof that it works.

— Bruno

You only stop where a person is needed: approve, paste into the environment, run user acceptance and publish. Everything else is produced and validated for you.

THE PROBLEMS IT PREVENTS

- **Screens built before the problem is understood**

Brainstorm, requirements and an approved clickable prototype before the first formula.

- **Formulas that don't delegate and drop records**

Delegation patterns in the Canvas skill, and QA checks them before you test.

- **A flow with no error handling**

Try/Catch, per-action authorization, logging and a 4-field response; the screen calls it inside `IfError`.

- **A user from one unit seeing another unit's data**

The block lives in the flow and the procedure (or in the security role); the screen only filters. The test denies by role and by unit.

START NOW

```
# inside Claude Code
/plugin marketplace add stage2dev/power-platform-skills
/plugin install pp-en@power-platform-kit

# in an empty folder
/pp-en:new an app to track orders across units
```

Type `/pp-en:` and the commands show up. At the end of each stage, follow the **Next step** block, in a fresh session (`/clear`). The `pp-en` skills are rolling out; until then, the pt-BR edition `pp` runs the same pipeline.

HOW IT WORKS

Ten stages, one session per stage

Each stage checks the previous one and closes by naming the next command. Everything is saved to disk, so the next one starts clean. Lost the thread? `/pp-en:progress` shows where the project stands.

#	COMMAND	WHAT HAPPENS	YOU
● Product definition			
1	<code>/pp-en:new</code>	Creates the project (git, config, <code>STATE.md</code>) and keeps the idea as it came: a sentence, a list or a screenshot.	Picks folder, commits and the model profile.
2	<code>/pp-en:brainstorm</code>	Requirements, business rules and the MVP cut in <code>prd.md</code> , in one of four conversation modes.	Answers and decides what goes into the MVP.
● Identity and experience			
3	<code>/pp-en:design</code>	Colors, fonts, components and navigation (five patterns), with a visual sample checked by screenshot.	Approves the identity.
4	<code>/pp-en:mockups</code>	Inventory of every screen, including loading, error, empty and no access; optional images.	Checks the screen list.
5	<code>/pp-en:prototype</code>	Clickable HTML prototype, with live role and navigation switching.	Clicks through and approves, or asks for adjustments.
● Build on the Power Platform			
6	<code>/pp-en:architecture</code>	Data track (SQL Server or Dataverse), permissions, the app↔flow contract and the build queue.	Chooses the track; ● creates the tables.
7	<code>/pp-en:build</code>	Canvas and Power Automate agents in parallel: <code>.pa.yaml</code> screens and flow JSON ready to paste.	● Pastes into Studio and the designer.
● Validation and delivery			
8	<code>/pp-en:test</code>	Validators for every layer and a test script in the environment, denying by role and by unit.	● Runs the script.
9	<code>/pp-en:uat</code>	User acceptance testing with real users.	● Leads the UAT.
10	<code>/pp-en:publish</code>	Production, user manual, technical guide and go-live checklist.	● Publishes.

● Only a person does it in the environment: Claude shows the walkthrough and waits for "done" or the error.

THE LOOPS

Prototype not approved

The adjustments go back to design; mockups and prototype redo only what changed.

→ `/pp-en:design`

Failing test

Each failure becomes a fix item, tagged as app or flow; then the test runs again.

→ `/pp-en:build app | flows`

App already live

The list of changes becomes one spec per front, the agents work in parallel and a new version ships.

→ `/pp-en:change`

THE METHOD

Who thinks is not who executes

Thinking and judging is the small part of the work; writing screens, flows and procedures is the big one. That's why each stage splits the two roles, and nothing reaches you without being checked.

THE HEAD

The stage's session

Talks with you, writes each agent's request (exact files, what "done" means, the command that proves it) and judges what comes back, running the validator again.

THE HANDS

The agents

Do only what the request says and close with: how they verified it (command and output), what they delivered, what they find risky and how confident they are.

● Accepted

Request delivered and proven. The stage moves on.

● Revision

The same agent, with a sharper request. Two at most.

● Escalated

It needs your decision, or two revisions didn't close it. It comes to you.

THE AGENTS

AGENT	WHERE IT COMES IN	WHAT IT DOES	HOW IT PROVES IT
Brainstorm · Designer	stages 2 and 3	Run in the session: they need to talk with you.	Your approval
mockups-agent	stage 4	Every screen, with loading, error and empty.	Spec checked offline
prototype-agent	stage 5	Clickable prototype on the approved design.	verificar-prototipo.py
architecture-agent	stage 6	Data model, permissions, contract and queue.	Judged by the session
sql-agent	stage 6	The spec's procedures, one per group, in parallel.	lint-procedure.py
canvas-agent	stage 7	Screens in pasteable YAML, up to 3 per agent.	validar-telas.py
automate-agent	stage 7	Flows in pasteable JSON, up to 3 per agent.	verificar-fluxo.py
qa-agent	stage 8	Read-only: contract, delegation, authorization.	Every validator
research-agent	on demand	Reads the project and official docs only; nothing goes to the web.	Finding with a source

MODEL PROFILES, CHOSEN IN /PP-EN:NEW

PROFILE	STAGE SESSION	ARCHITECTURE AND QA	SCREENS AND FLOWS	RESEARCH
Balanced RECOMMENDED	Opus	Opus	Sonnet	Sonnet
Maximum	Fable (or Opus)	Opus	Opus	Sonnet
Economy	Sonnet	Sonnet	Sonnet	Haiku
Inherit	The model the session opens with, for every role			

It also works on what's already live

YOU SAY	WHAT CLAUDE DOES
"The KPI is off" / "it's slow"	Walks screen → formula → source → flow → procedure → data, one hypothesis at a time, and proves the root cause before proposing code.
"They see another unit's data"	Checks the layer that actually blocks access: flow + procedure on SQL, or security roles on Dataverse.
"Audit the whole app"	Reviewers in parallel by discipline (UX, development, performance, data, flows, SQL), with the strongest findings checked again.
A list of changes	/pp-en: change: scope of each request, one spec per front, agents in parallel, QA and user acceptance of a new version.
"Promote to UAT/PROD"	Solutions, environment variables, connection references and the pac CLI; says what goes by pasting and what goes by solution.
"Is it ready?"	Runs the final gate: every validator that applies, plus the checklist.

CONVENTIONS THE KIT GUARANTEES

- ✓ **One data track per project:** SQL Server or Dataverse.
- ✓ **Writes always go through a flow,** with a 4-field response and `.Run()` inside `OnError`.
- ✓ **One separator everywhere:** `,` separates and `;` chains, in the en-US formula bar and in pasted YAML.
- ✓ **What was pasted is the source of truth:** a generator never overwrites what's already pasted.
- ✓ **The real name wins:** formulas and flows use the names read from the environment, never the ones from the plan.
- ✓ **The flow reads the caller** from its own context and authorizes each action; a new parameter goes at the end.
- ✓ **No environment literals:** no server or GUID; environment variables and connection references.
- ✓ **Done requires evidence:** command, output and date.

OPEN SOURCE

Improve it, add to it, fix it

A rule doesn't match your day-to-day? A component is missing? Open an issue or send a pull request: I follow and review each one.

The skill standard is in `docs/PADRAO-SKILL.md`.

COMING NEXT

A full Power BI skill

At the same level as this one: star schema, Power Query M and DAX, with the same care for standards, validation and proof.

Real-world feedback helps set priorities: try it and tell me what broke.

REQUIREMENTS

- **Claude Code** with plugin support
- **Power Platform environment** to paste, test and publish
- **Python 3.10+** and `pip install pyyaml` (validators)
- **Optional:** `OPENAI_API_KEY` for image mockups; without the key, the stage goes on without images

REPOSITORY

github.com/stage2dev/power-platform-skills

SITE

stage2dev.github.io/power-platform-skills/en

To update later: `claude plugin marketplace update power-platform-kit` and `claude plugin update pp-en`.